Slide 1–12: সহজ বাংলায় ব্যাখ্যা

এই ১২টা স্লাইডের মূল বিষয় হলো **সমস্যাগুলো কতটা কঠিন**, সেটা অনুযায়ী ভাগ করা (P, NP, NP-Hard, NP-Complete)। এগুলো না বুঝলে পরের Approximation Algorithm বোঝা যায় না।

Slide 1: Learning Objectives

এই লেকচারে তুমি পাঁচটা জিনিস শিখবে:

- Non-deterministic algorithm কী
- NP-Hard আর NP-Complete এর সম্পর্ক কী
- Approximation algorithm কী এবং কেন দরকার
- কিছু বেসিক approximation algorithm

Slide 2: P Class

P Class মানে সেই সমস্যা, যেটা Polynomial time-এ সমাধান করা যায়।

Polynomial time মানে সময়টা n-এর কোনো পাওয়ার, যেমন n, n^2, n log n। input বাড়লেও সময় খুব বেশি বাড়ে না।

উদাহরণ:

- Insertion Sort → $O(n^2)$
- Merge Sort → O(n log n)
- Sequential Search → O(n)

এই সমস্যাগুলো কম্পিউটার সহজেই করতে পারে।

Slide 3: Polynomial vs Exponential

Time complexity অনুযায়ী অ্যালগরিদম দুই ভাগে ভাগ হয়:

Polynomial Time (সহজ) **Exponential Time (কঠিন)**
Sequential Search: n 0/1 Knapsack: 2^n

Polynomial Time (সহজ)	Exponential Time (কঠিন)
Binary Search: log n	Travelling Salesman: 2^n
Insertion Sort: n^2	Graph Coloring: 2^n
Merge/Quick Sort: n log n	

একটা তুলনা: $n = 50$ হলে $n^2 = 2500$ ধাপ, কিন্তু 2^{50} প্রায় ১ লক্ষ কোটি ধাপের বেশি। এই কারণেই Exponential খুব ভয়ংকর।

স্লাইডের শেষ কথা: আমরা সবসময় Exponential সমস্যাকে Polynomial-এ নামিয়ে আনতে চাই। না পারলে Non-Deterministic Algorithm (NP Class) লিখি।

Slide 4: NP Class

NP মানে **Non-deterministic Polynomial time**। এর মূল কথা:

- সমাধান **খুঁজে বের করা** কঠিন, অর্থাৎ Polynomial time-এ করা যায় না (বা আমরা কোনো উপায় জানি না)।
- কিন্তু সমাধান **হাতে থাকলে** সেটা ঠিক না ভুল, তা Polynomial time-এ **যাচাই (check)** করা যায়।

উদাহরণ: **Sudoku game**।

Slide 5–6: Sudoku উদাহরণ

Sudoku-তে প্রতিটা row আর column-এ ১ থেকে ৯ কোনো repeat ছাড়া বসাতে হয়।

- **Slide 5:** অর্ধেক ফাঁকা ঘর। এটা সমাধান করতে অনেক চেষ্টা-ভুল লাগে, তাই time complexity Exponential।
- **Slide 6:** পুরো ভরা Sudoku। এবার প্রতিটা row, column আর box দেখলেই বোঝা যায় ঠিক আছে কিনা। এই যাচাই খুব দ্রুত, Polynomial time-এ হয়।

সহজ কথা: পাজল সমাধান করা কঠিন, কিন্তু সমাধান দেখে ঠিক কিনা বলা সহজ। পরীক্ষার খাতা লেখা কঠিন, কিন্তু উত্তরপত্র দেখে মিলিয়ে নম্বর দেওয়া সহজ।

স্লাইডের প্রশ্ন: "Why it is known as Non-Deterministic?" এর উত্তর হলো, সমাধান বের করার নির্দিষ্ট কোনো দ্রুত ধাপ আমাদের জানা নেই, কিন্তু পেয়ে গেলে যাচাই করা যায়।

Slide 7: Non-Deterministic Algorithm

- **Deterministic algorithm:** প্রতিটা statement কীভাবে কাজ করে তা পরিষ্কার জানা।
- **Non-deterministic algorithm:** বেশিরভাগ statement পরিষ্কার, কিন্তু কিছু statement কীভাবে কাজ করে তা আমরা জানি না। ভবিষ্যতে কেউ হয়তো বের করবে।

উদাহরণ: Binary Search-এর সময় O(log n)। আমরা চাই এর চেয়েও কম, অর্থাৎ Constant time O(1)। সেটা কীভাবে করা যায় জানি না। তাই একটা "রহস্যময় ধাপ" ধরে নিয়ে Non-deterministic algorithm লিখি।

Slide 8: Non-Deterministic Searching

কোডটা এরকম:

```
Search_NP(key)
{
   array list[100];
   i = check_array(key);      // O(1) – কীভাবে কাজ করে জানি না (যাদু!)
   if (list[i] == key)        // O(1)
       Found at index i       // O(1)
   else
       Not Found              // O(1)
}
```

`check_array(key)` নামের একটা ধাপ সরাসরি সঠিক index দিয়ে দেয়। কোনো তুলনা বা খোঁজাখুঁজি করতে হয় না (স্লাইডের লাল লেখা: "No comparing, searching")। তাই মোট সময় O(1)। কিন্তু এই ধাপটা কীভাবে কাজ করে তা আমরা জানি না, তাই এটা Non-Deterministic।

Slide 9: NP-Complete সমস্যার সারকথা

- NP-Complete সমস্যার জন্য Optimal solution বের করার কোনো Polynomial time algorithm নেই।
- অ্যালগরিদমগুলো Exponential, input বাড়লে সময় আর মেমোরি বিশাল হয়ে যায়।
- তাই এই সমস্যাগুলোর সমাধান খুব কঠিন।

Slide 10: Reduction (NP-Hard বোঝার আগের ধাপ)

ধরো দুটো সমস্যা আছে:

- **P1:** এর অ্যালগরিদম আমাদের নেই (A??)
- **P2:** এর অ্যালগরিদম আছে (B)

যদি P1-কে P2-এর আকারে বদলে P2-এর অ্যালগরিদম দিয়ে সমাধান করা যায়, তাহলে P1-এর জন্য আলাদা অ্যালগরিদম লেখার দরকার নেই। একে বলে **P1 is reducible to P2**, লেখা হয় $P1 \propto P2$। (আসল সংকেত $\leq_p$, স্লাইডে ∞-এর মতো দেখাচ্ছে।)

শর্ত: এই রূপান্তরের খরচও Polynomial time-এ হতে হবে।

উদাহরণ: ধরো তুমি জানো কীভাবে দুটো সংখ্যা গুণ করতে হয়। কেউ বর্গ (square) বের করতে বলল। তুমি একই সংখ্যাকে নিজের সাথে গুণ করে দিলে। এখানে "বর্গ" সমস্যাটা "গুণ" সমস্যায় reduce হয়েছে।

Slide 11: NP-Hard

একটা সমস্যা NP-Hard হবে যদি NP-এর সব সমস্যাকে Polynomial time-এ তার মধ্যে reduce করা যায়। সমস্যাটা নিজে NP-তে না থাকলেও চলবে।

ছবিতে NP-এর A, B, C, D সবাই X-এ reduce হচ্ছে ($A \propto X$, $B \propto X$…)। X হলো NP-Hard। এর মানে X সবার চেয়ে কঠিন বা অন্তত সমান কঠিন। স্লাইডে বলা আছে, base problem হিসেবে **Satisfiability (SAT)** নেওয়া হয়।

Slide 12: NP-Complete

যে সমস্যা একই সাথে NP এবং NP-Hard, সেটাই NP-Complete।

ভেন ডায়াগ্রামে দুই বৃত্তের **মাঝখানের অংশ** হলো NP-Complete।

---

১২ স্লাইডের সারসংক্ষেপ

Class	মানে	উদাহরণ

Class	মানে	উদাহরণ
P	Polynomial time-এ সমাধান করা যায়	Sorting, Searching
NP	সমাধান খোঁজা কঠিন, কিন্তু যাচাই সহজ	Sudoku
NP-Hard	NP-এর সব সমস্যার চেয়ে কঠিন (বা সমান)	SAT-ভিত্তিক সমস্যা
NP-Complete	NP ∩ NP-Hard (দুটোতেই আছে)	TSP, Knapsack, Vertex Cover

একটা সিঁড়ির মতো ভাবো: সহজ (P) → যাচাই সহজ কিন্তু খোঁজা কঠিন (NP) → সবচেয়ে কঠিন (NP-Hard) → মাঝখানে দুটোর মিলন (NP-Complete)।

বড় প্রশ্ন: NP-Complete সমস্যার নিখুঁত সমাধান খুব ধীর, তাহলে কী করব? এর উত্তরই পরের স্লাইডগুলোর **Approximation Algorithm**, অর্থাৎ নিখুঁত না হোক, কাছাকাছি সমাধান দ্রুত বের করা।

Slide 13–25: সহজ বাংলায় ব্যাখ্যা

আগের অংশে শিখেছ NP-Complete সমস্যা খুব কঠিন। এখন প্রশ্ন হলো, **এগুলো নিয়ে আমরা করব কী?** এই অংশে তার উত্তর আছে।

Slide 13: Optimization Problem

Optimization Problem মানে সব সম্ভাব্য সমাধানের মধ্যে সবচেয়ে ভালোটা খুঁজে বের করা। লক্ষ্য হতে পারে সবচেয়ে কম (minimum) অথবা সবচেয়ে বেশি (maximum)।

বাস্তবের অনেক optimization সমস্যাই NP-Hard বা NP-Complete। তাই সব input-এর জন্য নিখুঁত সমাধান Polynomial time-এ দেওয়ার অ্যালগরিদম বানানো সম্ভব না।

তিনটা উদাহরণ:

- **TSP (Travelling Salesman):** সব শহর ঘুরে আসার সবচেয়ে ছোট রাস্তা বের করা → **minimize**
- **0/1 Knapsack:** ব্যাগে জিনিস ভরে সবচেয়ে বেশি লাভ করা → **maximize**
- **Vertex Cover:** সবচেয়ে কম সংখ্যক vertex বেছে সব edge ঢেকে ফেলা → **minimize**

Slide 14: NP-Complete সমস্যা কীভাবে সমাধান করব?

তিনটা উপায় আছে। একটা টেবিলে দেখলে সহজ:

উপায়	সময়	সমাধানের মান
Brute-force	কোনো গ্যারান্টি নেই (অনেক ধীর)	☑ নিখুঁত (optimal)
Heuristic	☑ Polynomial time-এ হবে	কোনো গ্যারান্টি নেই (ভালো হতেও পারে, খারাপও)
Approximation	☑ Polynomial time-এ হবে	☑ optimal-এর কাছাকাছি হবে

Approximation-ই সবচেয়ে ভালো ভারসাম্য: দ্রুতও, আবার কাছাকাছি সঠিকও।

Slide 15: Brute-force Algorithm

Brute-force মানে **সবকিছু একে একে চেষ্টা করে দেখা।**

TSP-তে: সব সম্ভাব্য রাস্তা লিখে ফেলো, প্রতিটার দূরত্ব হিসাব করো, তারপর সবচেয়ে ছোটটা বেছে নাও। উত্তর নিখুঁত হবে, কিন্তু শহর বেশি হলে এত রাস্তা হয় যে কম্পিউটারও হাঁপিয়ে যায়।

উদাহরণ: ১০০টা তালাচাবির মধ্যে সঠিক চাবিটা খুঁজতে একটা একটা করে সবগুলো লাগিয়ে দেখা।

Slide 16: Heuristic Technique

Heuristic মানে **শর্টকাট বা আন্দাজের পদ্ধতি।** সমাধান সবসময় সেরা হয় না, তবে সীমিত সময়ে কাজ চালানোর মতো হয়।

এতে ব্যবহার হয় অভিজ্ঞতা, আন্দাজ, ট্রায়াল-এন্ড-এরর, বাদ দিতে দিতে এগোনো, আর আগের ডেটা দেখা। যখন নিখুঁত সমাধান অসম্ভব বা অব্যবহারিক, তখন এটা কাজে লাগে।

Slide 17: Heuristic-এর উদাহরণ (ফাস্ট ফুড)

- Fast Food ABC বাংলাদেশে ব্যবসা শুরু করল, আর তাদের স্টক প্রাইস বেড়ে গেল।

- এক বিশ্লেষক ভাবল, "বাংলাদেশ তো সব ফাস্ট ফুডের জন্য লাভজনক।"
- পরের বছর Fast Food XYZ বাংলাদেশে আসছে শুনে সে কিছু না দেখেই বলে দিল "কিনে ফেলো।"

এতে সময় বাঁচল, কিন্তু সিদ্ধান্ত ভুলও হতে পারে। হয়তো XYZ-এর খাবার বাংলাদেশিদের পছন্দ হবে না। **এটাই heuristic-এর ঝুঁকি: দ্রুত, কিন্তু নিশ্চয়তা নেই।**

Slide 18: TSP-তে Heuristic (Nearest Neighbour)

এটা সবচেয়ে সহজ TSP heuristic:

1. যেকোনো একটা শহর থেকে শুরু করো।
2. সবচেয়ে কাছের **না-যাওয়া** শহরে যাও।
3. এভাবে সব শহর শেষ হলে **প্রথম শহরে ফিরে এসো।**

উদাহরণ: ঢাকা থেকে শুরু করে প্রতিবার সবচেয়ে কাছের শহরে যাওয়া। এতে রাস্তা ছোট হতেও পারে, বড়ও হতে পারে। কোনো গ্যারান্টি নেই।

Slide 19: Approximation Algorithm

Approximation algorithm এমন সমাধান দেয় যা **প্রমাণসহ optimal-এর কাছাকাছি।** Heuristic-এর সাথে মূল পার্থক্য এটাই: heuristic-এ ভালো হবে কিনা জানা নেই, কিন্তু approximation-এ **গাণিতিকভাবে প্রমাণ করা থাকে** যে উত্তর সর্বোচ্চ এতটা খারাপ হতে পারে।

কখন ব্যবহার হয়: যখন optimal সমাধান বের করা অসম্ভব (intractable), অথবা নিখুঁত উত্তর দরকার নেই কিন্তু কাছাকাছি হলেই চলে।

চ্যালেঞ্জ : আমরা optimal মানটাই জানি না, তবু প্রমাণ করতে হবে যে আমাদের উত্তর তার কাছাকাছি। এটা কঠিন অংশ।

Slide 20: Definition

কোনো optimization problem P-এর জন্য অ্যালগরিদম A হলো approximation algorithm, যদি যেকোনো input I-এর জন্য সে একটা **feasible (বৈধ) সমাধান** দেয়। এর মানে সমাধানটা সেরা না হলেও নিয়ম মেনে বৈধ হতে হবে।

Given an optimization problem $\mathcal{P}$, an algorithm $\mathcal{A}$ is said to be an *approximation algorithm* for $\mathcal{P}$, if for any given instance I, it returns an approximate solution, that is a feasible solution.

Slide 21: Approximation-এর ধরন

সংকেতগুলো আগে বুঝে নাও:

- **A*(I)** = optimal মান (সবচেয়ে ভালো)
- **A(I)** = আমাদের অ্যালগরিদমের পাওয়া মান

১. Absolute approximation: optimal আর আমাদের উত্তরের **পার্থক্য** সবসময় একটা ধ্রুবক k-এর বেশি হবে না, অর্থাৎ $|A^*(I) - A(I)| \leq k$। উদাহরণ: Planar graph coloring।
যেমন: "আমার উত্তর সেরা উত্তরের চেয়ে সর্বোচ্চ ১ বেশি হবে।"

২. Relative approximation: optimal আর আমাদের উত্তরের **অনুপাত (ratio)** একটা ধ্রুবক k-এর বেশি হবে না। উদাহরণ: Vertex Cover।
যেমন: "আমার উত্তর সেরা উত্তরের সর্বোচ্চ ২ গুণ হবে।"

Slide 22: Approximation Ratio ρ(n)

- **C*** = optimal সমাধানের cost
- **C** = আমাদের অ্যালগরিদমের সমাধানের cost

$\rho(n) \geq \max(C/C^*, C^*/C)$

Minimization সমস্যায় (যেমন Vertex Cover) $C \geq C^*$, তাই ratio = C/C^*। Maximization সমস্যায় (যেমন Knapsack) $C \leq C^*$, তাই ratio = C^*/C। max নেওয়ায় ratio সবসময় **১ বা তার বেশি** থাকে।

যে অ্যালগরিদমের ratio $\rho(n)$, তাকে বলে **ρ(n)-approximation algorithm**। ρ যত ১-এর কাছে, অ্যালগরিদম তত ভালো। স্লাইডের নিচের লেখা ($A(I)=C$, $A^*(I)=C^*$, $K=\rho(n)$) শুধু আগের সংকেতগুলো মিলিয়ে দেখানো।

Slide 23: সংখ্যা দিয়ে উদাহরণ

- একটা graph-এর **MST (Minimum Spanning Tree)**-এর ওজন 10 (এটাই optimal)।
- আমাদের অ্যালগরিদম যে spanning tree বানায়, তার ওজন সবসময় 15-এর কম।
- **Approximation ratio = 15/10 = 1.5**

মানে আমাদের উত্তর সর্বোচ্চ ১.৫ গুণ খারাপ। ১০০ টাকার কাজ আমরা সর্বোচ্চ ১৫০ টাকায় করছি।

Slide 24: Approximation-এর উদাহরণ

১. **Vertex Cover:** optimization হলো সবচেয়ে কম vertex-এর cover খোঁজা। approximation হলো কম vertex-এর একটা cover খোঁজা।

২. **TSP:** optimization হলো সবচেয়ে ছোট cycle খোঁজা। approximation হলো ছোট একটা cycle খোঁজা।

৩. **Set Cover:** সম্পদ ভাগ করার সমস্যা। এখানে logarithmic approximation ratio ব্যবহার হয়।

৪. **Subset Sum:** কিছু সংখ্যার একটা subset বাছো, যার যোগফল target t-এর **সমান বা কম**, কিন্তু যতটা সম্ভব বড়।

সহজ কথা: "সবচেয়ে ভালো" জিনিসটা খুঁজতে না পারলে "যথেষ্ট ভালো" জিনিসটা খুঁজি, আর প্রমাণ করি সেটা কতটা ভালো।

Slide 25: এই লেকচারের তিন উদাহরণ

১. **Vertex Cover**
২. **Traveling Salesman Problem**
৩. **Bin Packing**

পরের স্লাইডগুলোতে (২৬ থেকে ৬৩) এই তিনটা বিস্তারিত আছে।

১৩–২৫ এর সারসংক্ষেপ

পুরো গল্পটা এক লাইনে: NP-Complete সমস্যা নিখুঁতভাবে সমাধান করতে অনেক সময় লাগে (Brute-force)। আন্দাজে সমাধান করলে (Heuristic) সময় কম লাগে, কিন্তু মান নিশ্চিত না। তাই **Approximation** নিই, যেখানে সময়ও কম লাগে, আবার প্রমাণ করা যায় উত্তর optimal-এর কতটা কাছে।

পরীক্ষার জন্য মনে রাখো:

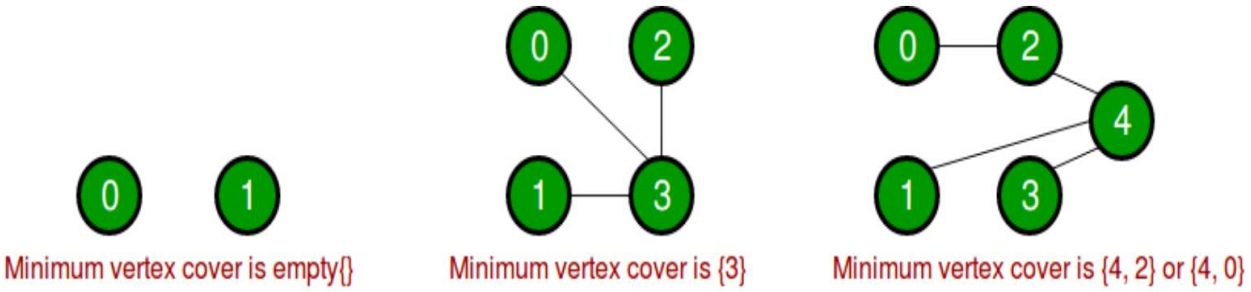
- Brute-force = নিখুঁত কিন্তু ধীর
- Heuristic = দ্রুত কিন্তু মানের গ্যারান্টি নেই
- Approximation = দ্রুত এবং মানের গ্যারান্টি আছে
- Approximation ratio = $\rho(n) = \max(C/C^*, C^*/C)$, সবসময় ≥ 1

Slide 26–41: Vertex Cover (সহজ বাংলায়)

Slide 26: Vertex Cover কী?

Vertex Cover হলো একটা graph-এর কিছু vertex বেছে নেওয়া, যাতে **প্রতিটা edge-এর অন্তত একটা মাথা (vertex) সেই বাছাই করা সেটে থাকে।** নাম Vertex Cover হলেও আসলে এই vertex-গুলো দিয়ে **সব edge ঢেকে ফেলা** হয়।

সমস্যা: সবচেয়ে **ছোট** (minimum) vertex cover খুঁজে বের করো।



ছবির উদাহরণ:

- কোনো edge নেই → minimum cover খালি `{}`
- 3 নম্বর vertex সবার সাথে যুক্ত → minimum cover `{3}`
- আরেকটায় `{4, 2}` বা `{4, 0}`

সহজ তুলনা: একটা এলাকার সব রাস্তা পাহারা দিতে হবে। রাস্তার মোড়ে (vertex) পুলিশ বসালে ওই মোড়ে যুক্ত সব রাস্তা পাহারা হয়। প্রশ্ন হলো, **সবচেয়ে কম কয়টা মোড়ে পুলিশ বসালে সব রাস্তা পাহারায় থাকবে?**

Slide 27: Formal সংজ্ঞা

এখানে কঠিন কিছু নেই।

আমাদের কাছে আছে:

G = (V, E)

যেখানে:

- `V` = সব vertex
- `E` = সব edge

আমরা `V` থেকে কিছু vertex বেছে নেব।

ধরি:

`C = আমাদের বাছাই করা vertex`

C তখনই Vertex Cover হবে যখন:

প্রতিটি edge-এর অন্তত একটি endpoint C-এর মধ্যে থাকবে।

আর আমরা চাই:

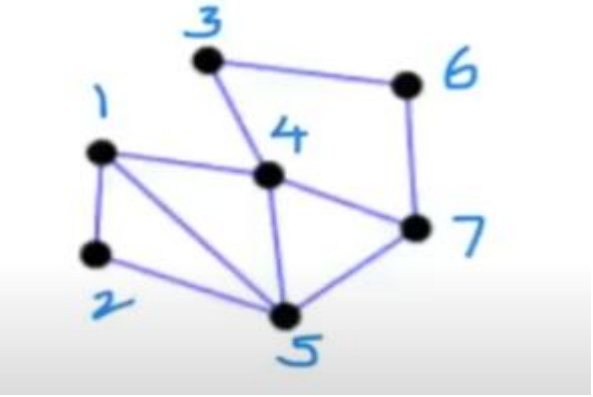
`|C| যত কম সম্ভব`

অর্থাৎ:

Minimum Vertex Cover = সবচেয়ে কম vertex দিয়ে সব edge cover করা।

Slide 28–32: উদাহরণ গ্রাফ

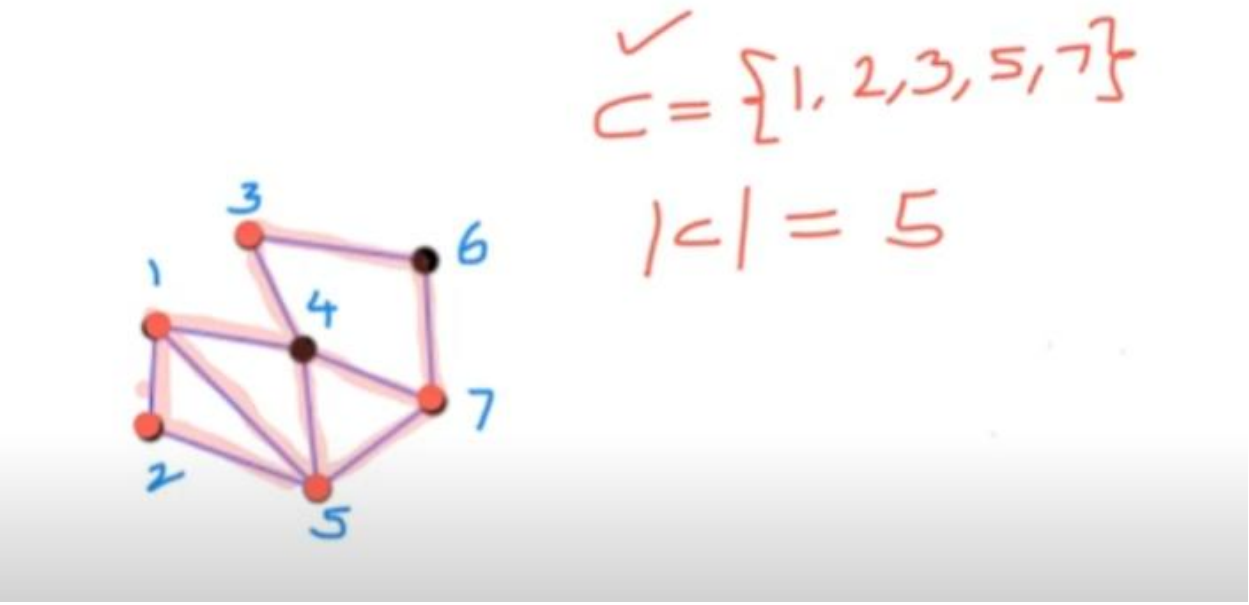
গ্রাফে ৭টা vertex আর ১০টা edge আছে:



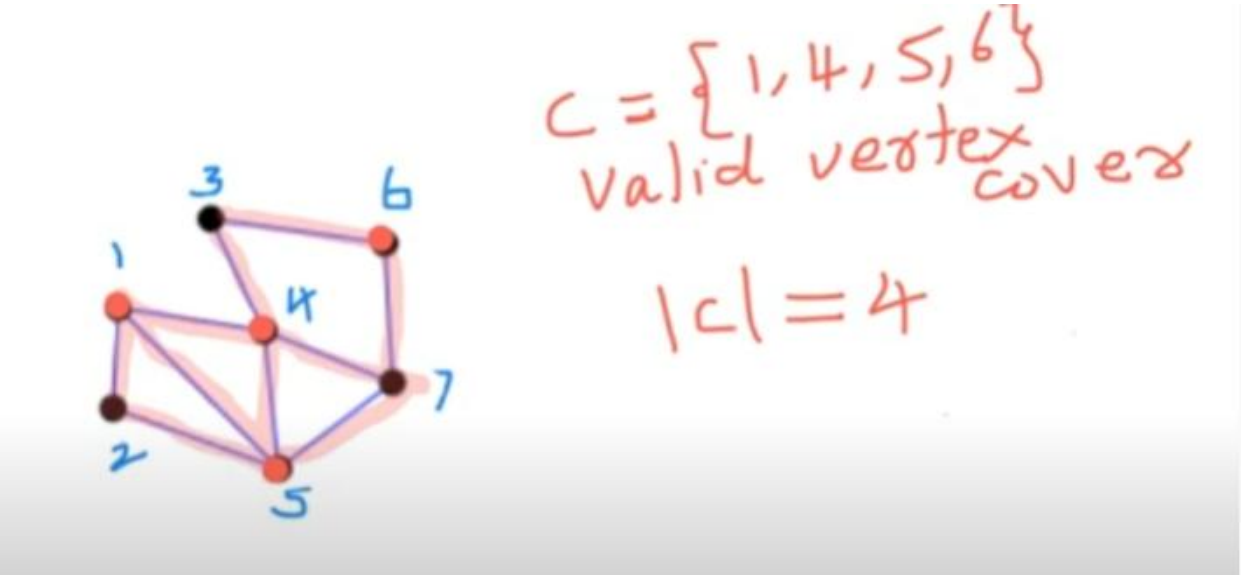
E = {(1,2), (1,5), (1,4), (2,5), (3,4), (3,6), (4,5), (4,7), (5,7), (6,7)}

চেষ্টা ১: C = {1, 4, 7} ✗ এটা cover **না**।
কারণ (3,6) edge-এর কোনো মাথাই {1,4,7}-এ নেই, আবার (2,5)-ও বাদ পড়ে গেছে। স্লাইডে এই দুটো edge সবুজ দাগে দেখানো।

চেষ্টা ২: C = {1, 2, 3, 5, 7} ☑ সব edge cover হয়েছে, |C| = 5।



চেষ্টা ৩: C = {1, 4, 5, 6} ☑ সব edge cover হয়েছে, |C| = **4**।
স্লাইডে বলা হয়েছে, এই গ্রাফে ৪-এর কম দিয়ে cover হয় না, তাই **minimum vertex cover = 4**।



Slide 32-এর ছবিতে দেখানো হয়েছে, সব vertex নিলেও (trivial cover) cover হয়, কিন্তু সেটা সবচেয়ে ছোট না।

মনে রাখো: cover এক রকম না, অনেক রকম হতে পারে। আমাদের লক্ষ্য সবচেয়ে ছোটটা খোঁজা।

Slide 33: Brute-force কেন কাজ করে না?

- n টা vertex থাকলে সম্ভাব্য subset আছে 2^n টা।
- প্রতিটা subset দেখে বৈধ cover বেছে সবচেয়ে ছোটটা নিতে হবে।
- তাই সময় $O(2^n)$, যা exponential। n বড় হলে কম্পিউটারের মাস বা বছর লেগে যাবে।

উপসংহার: তাই একমাত্র পথ **approximation**।

Slide 34–35: Approximation Algorithm (APPROX-VERTEX-COVER)

অ্যালগরিদমটা খুব সহজ:

1. C = খালি সেট
2. E' = সব edge
3. যতক্ষণ E'-এ edge আছে:

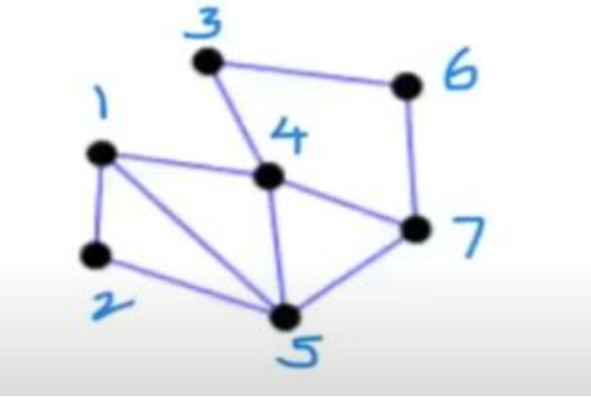
- যেকোনো একটা edge (u, v) তুলে নাও
- **u এবং v দুটোকেই** C-তে ঢোকাও
- u বা v-এর সাথে যুক্ত **সব edge** E' থেকে মুছে ফেলো

4. C ফেরত দাও

সময়: O(V + E)। যেহেতু E সর্বোচ্চ n^2 হতে পারে, তাই $O(n + n^2) = \mathbf{O(n^2)}$, যা **Polynomial time**। Brute-force-এর $O(2^n)$ এর তুলনায় অনেক দ্রুত।

Slide 36–39: ধাপে ধাপে চালিয়ে দেখা

শুরু: E' = {(1,2), (1,5), (1,4), (2,5), (3,4), (3,6), (4,5), (4,7), (5,7), (6,7)}



ধাপ	তোলা edge	C	মোছা হলো	বাকি E'
১	(1,2)	{1,2}	1 বা 2 যুক্ত সব edge	(3,4),(3,6),(4,5),(4,7),(5,7),(6,7)
২	(5,7)	{1,2,5,7}	5 বা 7 যুক্ত সব edge	(3,4),(3,6)
৩	(3,6)	{1,2,5,7,3,6}	3 বা 6 যুক্ত সব edge	**খালি** ☑

ফলাফল: C = {1, 2, 5, 7, 3, 6}, |C| = **6**

আসল optimal ছিল 4, আমরা পেলাম 6। নিখুঁত না, কিন্তু বৈধ cover এবং অনেক দ্রুত।

Slide 40: Maximal Matching

- **Matching:** এমন কিছু edge-এর সেট, যাদের মধ্যে **কোনো common vertex নেই।**

এটা খুব গুরুত্বপূর্ণ।

Matching

কিছু edge নেব, কিন্তু কোনো দুইটা edge-এর **common vertex থাকবে না।**

যেমন:

- `(1,2)`
- `(5,7)`
- `(3,6)`
- এখানে:
- `(1,2)`
- এর vertex 1,2
- `(5,7)`
- এর vertex 5,7
- `(3,6)`
- এর vertex 3,6
- কোনো vertex common নেই।
- তাই:
- `M = {(1,2),(5,7),(3,6)}`
- একটা **Matching**।
- এখানে মোট:
- `|M| = 3`

- **Maximal Matching:** এমন matching, যাতে আর কোনো edge যোগ করা যায় না।

আমাদের অ্যালগরিদম যে edge-গুলো তুলেছে, সেগুলো একটা matching:
M = {(1,2), (5,7), (3,6)}, |M| = 3

কেন common vertex নেই? প্রতিবার edge তোলার পর তার দুই মাথা যুক্ত সব edge মুছে ফেলা হয়েছে। তাই পরের edge-এ আগের কোনো vertex আসতে পারে না।

প্রতিটা edge থেকে দুটো করে vertex নেওয়া হয়েছে, তাই:
|C| = 2 × |M| → 6 = 2 × 3

Slide 41: প্রমাণ (পরীক্ষায় গুরুত্বপূর্ণ)

আমরা প্রমাণ করতে চাই: **|C| ≤ 2 × |C*|** (C* = optimal cover)

যুক্তি ধাপে ধাপে:

আমাদের দেখাতে হবে:

$|C| \leq 2|C^*|$

এখানে:

- C = আমাদের algorithm-এর answer
- C^* = আসল optimal answer

সহজভাবে:

আমাদের answer optimal answer-এর ২ গুণের বেশি হবে না।

Step 1: আমাদের নেওয়া edgeগুলো Matching

আমরা নিয়েছি:

```
(1,2)
(5,7)
(3,6)
```

কোনো common vertex নেই।

তাই:

$|M| = 3$

Step 2: Optimal solution-ও এই edgeগুলো cover করতে বাধ্য

Optimal solution C^* হলেও তাকে:

```
(1,2)
(5,7)
(3,6)
```

এই তিনটা edge cover করতেই হবে।

আর এগুলোর মধ্যে কোনো common vertex নেই।

তাই প্রতিটি edge-এর জন্য অন্তত একটি করে আলাদা vertex লাগবে।

অর্থাৎ:

$$|C^*| \geq |M|$$

আমাদের example-এ:

$$|C^*| \geq 3$$

আসলেই optimal ছিল 4।

Step 3: আমরা কত vertex নিয়েছি?

প্রতিটি edge থেকে আমরা **দুইটা vertex** নিয়েছি।

তাই:

$$|C| = 2|M|$$

আমাদের ক্ষেত্রে:

$$\begin{aligned} |C| &= 2 \times 3 \\ &= 6 \end{aligned}$$

Step 4: দুইটা equation একসাথে করি

আমরা জানি:

$$|C^*| \geq |M|$$

অর্থাৎ:

$$|M| \leq |C^*|$$

দুই পাশে 2 দিয়ে গুণ:

$$2|M| \leq 2|C^*|$$

কিন্তু:

$$|C| = 2|M|$$

তাই:

$$\star \quad |C| \leq 2|C^*|$$

ব্যস!

এটাই প্রমাণ করে যে algorithm-টা:

2-Approximation Algorithm

সংখ্যায় মিলিয়ে দেখো: $|C| = 6$ আর $|C^*| = 4$, তাই $2 \times 4 = 8$। $6 \leq 8$ ☑

সহজ কথায় যুক্তি: ধরো ৩টা আলাদা (একে অপরের সাথে সম্পর্কহীন) edge আছে। সেরা সমাধানকেও অন্তত ৩টা vertex নিতেই হবে, প্রতিটা edge-এর জন্য একটা করে। আমরা প্রতিটা edge থেকে দুটো করে নিই, মানে ৬টা। তাই আমরা optimal-এর **সর্বোচ্চ দ্বিগুণ** নিচ্ছি, এর বেশি কখনো না।

Slide 42–56: TSP (Travelling Salesman Problem)

Slide 42: Hamiltonian Cycle

Hamiltonian cycle হলো এমন একটা চক্র (cycle) যা **প্রতিটা vertex ঠিক একবার** ছুঁয়ে শুরুর জায়গায় ফিরে আসে।

- Hamiltonian cycle-এর যেকোনো একটা edge মুছে দিলে একটা **Hamiltonian path** পাওয়া যায়।
- গ্রাফে Hamiltonian cycle/path আছে কিনা, এই সিদ্ধান্ত নেওয়াটাই **NP-Complete**।

TSP-র সাথে সম্পর্ক: একজন সেলসম্যান সব শহর একবার করে ঘুরে বাড়ি ফিরবে। প্রশ্ন হলো, **সবচেয়ে কম দূরত্বের** ভ্রমণপথ কোনটা? মানে সবচেয়ে সস্তা Hamiltonian cycle বের করা।

Slide 43–46: Spanning Tree ও MST (আগে এটা বুঝতে হবে)

ধরো ৪টা শহর আছে:



Spanning Tree-এর ৩টা নিয়ম:

1 সব vertex থাকতে হবে

A, B, C, D সবাই থাকবে।

2 কোনো cycle থাকবে না

মানে এমন হবে না:

```
A → B → C → A
```

3 nটা vertex হলে n-1টা edge

যদি:

```
n = 4
```

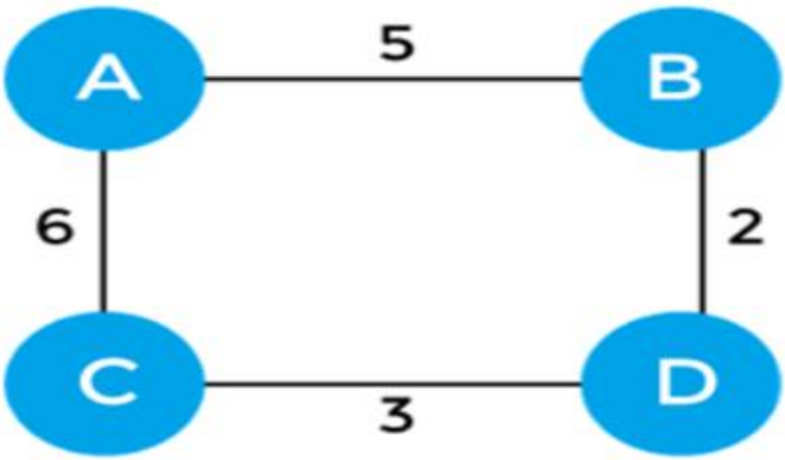
তাহলে spanning tree-তে:

```
4 - 1 = 3 edges
```

থাকবে।

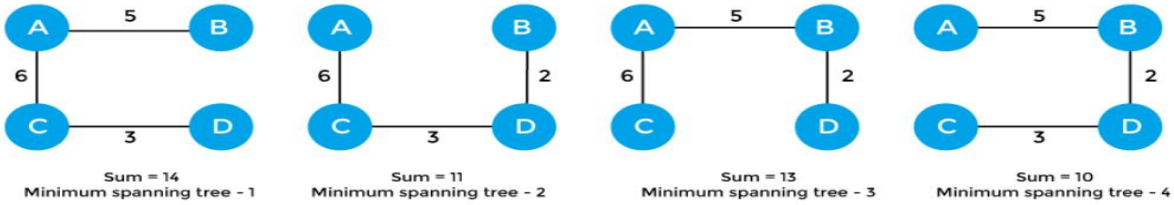
উদাহরণ: ৫টা vertex হলে spanning tree-তে edge থাকবে ৪টা। (Complete গ্রাফে $n^{(n-2)}$ টা spanning tree হতে পারে। n = 5 হলে $5^3 = 125$টা।)

Minimum Spanning Tree (MST): সব spanning tree-র মধ্যে যেটার **edge-এর ওজনের যোগফল সবচেয়ে কম।**

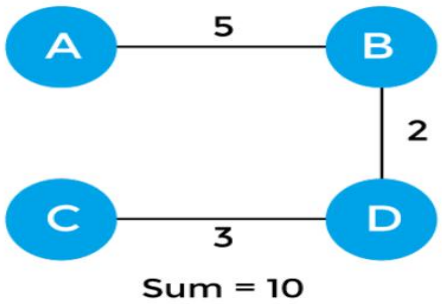


Weighted graph

The sum of the edges of the above graph is 16. Now, some of the possible spanning trees created from the above graph are -



So, the minimum spanning tree that is selected from the above spanning trees for the given weighted graph is -



স্লাইডের উদাহরণ: A–B = 5, A–C = 6, B–D = 2, C–D = 3

Spanning tree	যোগফল
১ম	14
২য়	11
৩য়	13
৪র্থ	**10** ☑

MST হলো ৪র্থটা: edge **A–B (5), B–D (2), D–C (3)**, যোগফল **10**।

সহজ তুলনা: ৪টা গ্রাম সংযুক্ত করতে সবচেয়ে কম খরচে রাস্তা বানাতে চাও, যাতে সব গ্রামে যাওয়া যায়। সেই সস্তা রাস্তার নকশাই MST।

Slide 47: TSP-র সমস্যা

TSP-তে আমরা এমন একটা graph নিই যেখানে:

- সব vertex-এর মধ্যে edge আছে
- প্রতিটি edge-এর একটা cost আছে

আমাদের লক্ষ্য:

সব vertex ঠিক একবার visit করে আবার শুরুতে ফিরে আসা এবং total cost যত কম সম্ভব রাখা।

যেমন:

```
A → B → C → D → E → A
```

এটা একটা tour।

আর যদি:

```
Cost = 100
```

হয়, তাহলে আমরা অন্য tour খুঁজব যেটার cost হয়তো 90, 80, 70...

সবচেয়ে কম যেটা পাওয়া যাবে সেটাই **Optimal TSP Tour**।

⭐ Triangle Inequality কী?

এটা TSP-এর approximation-এর জন্য খুব গুরুত্বপূর্ণ।

সহজভাবে:

সরাসরি যাওয়ার cost কখনো ঘুরে যাওয়ার cost-এর চেয়ে বেশি হবে না।

ধরো:

```
A → B = 5
B → C = 4
```

তাহলে:

```
A → C ≤ 9
```

হতে হবে।

কারণ A থেকে সরাসরি C যাওয়া যদি 12 হয়, কিন্তু A → B → C যেতে 9 লাগে, তাহলে সরাসরি যাওয়াটা বেশি।

Triangle inequality থাকলে:

```
Direct path ≤ Indirect path
```

খুব সহজে মনে রাখো:

Shortcut নিলে খরচ বাড়বে না।

এই কথাটাই পরে আমাদের proof-এ সবচেয়ে গুরুত্বপূর্ণ হবে।

উদাহরণ: ঢাকা থেকে চট্টগ্রাম সরাসরি যাওয়া ≤ ঢাকা → কুমিল্লা → চট্টগ্রাম। বাস্তব দূরত্বে এটা প্রায় সবসময় সত্যি।

Slide 48: মূল আইডিয়া (দুটো ধারণা)

এখন আসল trick শুরু।

আমরা জানি TSP-এর optimal answer বের করা কঠিন।

তাই আমরা বলি:

"চলো আগে MST বের করি।"

কারণ MST বের করা অনেক সহজ।

⭐ কেন MST ব্যবহার করব?

ধরো optimal TSP tour হলো:

```
A → B → C → D → E → A
```

এখানে একটা edge বাদ দিই:

```
E → A
```

তাহলে পাই:

```
A → B → C → D → E
```

এটা একটা **Spanning Tree**।

অর্থাৎ:

Optimal TSP tour থেকে একটা edge বাদ দিলে একটা spanning tree পাওয়া যায়।

আর MST হলো সব spanning tree-এর মধ্যে সবচেয়ে সস্তা।

তাই:

```
MST ≤ Optimal TSP
```

এটাই আমাদের **Lower Bound**।

🎯 Lower Bound মানে কী?

Lower Bound মানে:

এর চেয়ে কম হওয়া সম্ভব না।

ধরো:

```
MST cost = 10
```

তাহলে optimal TSP-এর cost কখনো 10-এর নিচে হতে পারবে না।

অর্থাৎ:

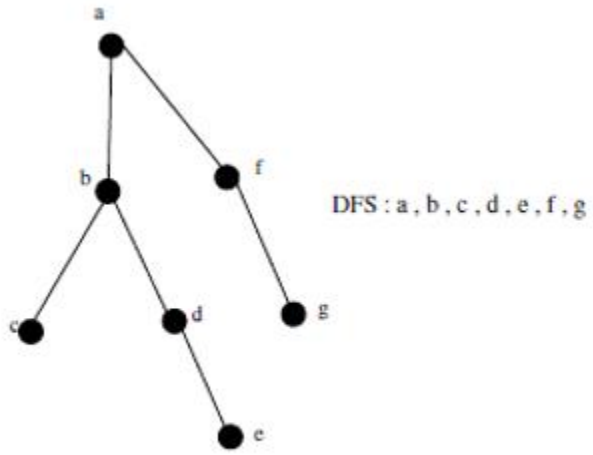
```
Optimal TSP ≥ 10
```

তাই MST আমাদের একটা ধারণা দেয়:

"Optimal answer অন্তত এতটুকু হবে।"

ধারণা ২: Pre-order walk। একটা tree-তে DFS করে vertex-গুলো প্রথম দেখামাত্র লিখে যাওয়া।

স্লাইডের tree-তে DFS order: **a, b, c, d, e, f, g**। আর ফিরে আসাসহ পুরো walk: a, b, c, b, d, e, d, b, a, f, g, f, a।



Pre–order walk : a , b , c , b, d , e, d , b, a , f , g, f, a

Pre-order Walk (একদম সহজ করে)

আগের ব্যাখ্যাটা খুব সংক্ষেপে ছিল, তাই এবার ধাপে ধাপে দেখাচ্ছি।

আগে tree-টা দেখো

স্লাইডের tree এরকম:

```
      a
     / \
    b   f
   / \   \
  c   d   g
      |
      e
```

- a-এর দুই child: b, f
- b-এর দুই child: c, d
- d-এর child: e
- f-এর child: g

Pre-order-এর একটাই নিয়ম

কোনো vertex-এ প্রথমবার পৌঁছালেই সাথে সাথে তার নাম লিখে ফেলো। তারপর তার child-দের কাছে যাও (বাঁ থেকে ডানে)। ফেরার সময় কিছু লিখো না।

উদাহরণ: কোনো আত্মীয়ের বাড়িতে ঢুকেই "এসেছি" বলে হাজিরা দিলে। তারপর ওই বাড়ির ভেতরের ঘরগুলোতে যাও। বাড়ি থেকে বেরোনোর সময় আর হাজিরা দিতে হয় না।

ধাপে ধাপে চালাও

ধাপ	কোথায় আছি	কী করলাম	লেখা তালিকা
১	a	শুরু, a লিখলাম	**a**
২	b	a-এর প্রথম child b-তে গেলাম, b লিখলাম	a, **b**
৩	c	b-এর প্রথম child c-তে গেলাম, c লিখলাম	a, b, **c**
৪	b	c-এর child নেই, তাই b-তে ফিরলাম (**লিখলাম না**)	a, b, c
৫	d	b-এর পরের child d-তে গেলাম, d লিখলাম	a, b, c, **d**
৬	e	d-এর child e-তে গেলাম, e লিখলাম	a, b, c, d, **e**
৭	d → b → a	e-র child নেই, d-র আর child নেই, b-র আর child নেই, তাই ফিরে ফিরে a-তে এলাম (**লিখলাম না**)	a, b, c, d, e
৮	f	a-এর পরের child f-এ গেলাম, f লিখলাম	a, b, c, d, e, **f**
৯	g	f-এর child g-তে গেলাম, g লিখলাম	a, b, c, d, e, f, **g**
১০	f → a	ফিরে এলাম (**লিখলাম না**)	a, b, c, d, e, f, g

Pre-order = a, b, c, d, e, f, g

Full walk-এর সাথে পার্থক্য

আগের উত্তরে দুটো তালিকা ছিল, সেগুলো এক জিনিস না:

	কখন লেখা হয়	ফল
Pre-order walk	শুধু **প্রথমবার** পৌঁছালে	a, b, c, d, e, f, g
Full walk	**যাওয়ার সময়ও, ফেরার সময়ও**	a, b, c, b, d, e, d, b, a, f, g, f, a

Full walk-এ ফেরার সময় যে vertex-এ আসছি, সেটাও লিখছি। তাই b, d, a, f বারবার এসেছে। এটা প্রতিটা edge **দুবার** পার হওয়া বোঝায়, তাই cost = 2 × MST।

TSP-র সাথে সম্পর্ক কী?

Tour-এ প্রতিটা শহর **একবারই** থাকতে হয়। কিন্তু full walk-এ repeat আছে, তাই সেটা বৈধ tour না। Repeat মুছে দিলেই pre-order পাওয়া যায়:

```
Full walk : a, b, c, b, d, e, d, b, a, f, g, f, a
                    ↑     (c → b → d)
Pre-order : a, b, c,    d, e,          f, g   → শেষে a-তে ফেরা
```

মানে c থেকে b হয়ে d-তে যাওয়ার বদলে **সরাসরি c → d** যাই। Triangle inequality-র কারণে এই shortcut-এ cost বাড়ে না।

চূড়ান্ত tour: a → b → c → d → e → f → g → a

Slide 49: অ্যালগরিদম (APPROX-TSP-TOUR)

এখন পুরো algorithm খুব সহজ।

Step 1

একটা vertex-কে Root বানাও।

ধরি:

A

Step 2

Graph-এর **MST** বের করো।

Step 3

MST-তে DFS/pre-order চালাও।

মানে:

কোনো vertex-কে প্রথমবার দেখার সময় তার নাম লিখে রাখো।

ধরো পেলাম:

```
A, B, C, H, D, E, F, G
```

Step 4

এখন এই order অনুসারে TSP tour বানাও:

```
A → B → C → H → D → E → F → G → A
```

ব্যস।

এটাই আমাদের approximate TSP solution।

Slide 50: Lower ও Upper Bound

Lower Bound

বলছে:

এর চেয়ে কম হতে পারবে না।

যেমন:

```
Optimal ≥ MST
```

Upper Bound

বলছে:

আমাদের solution এর চেয়ে বেশি হবে না।

TSP-তে আমরা পরে দেখাব:

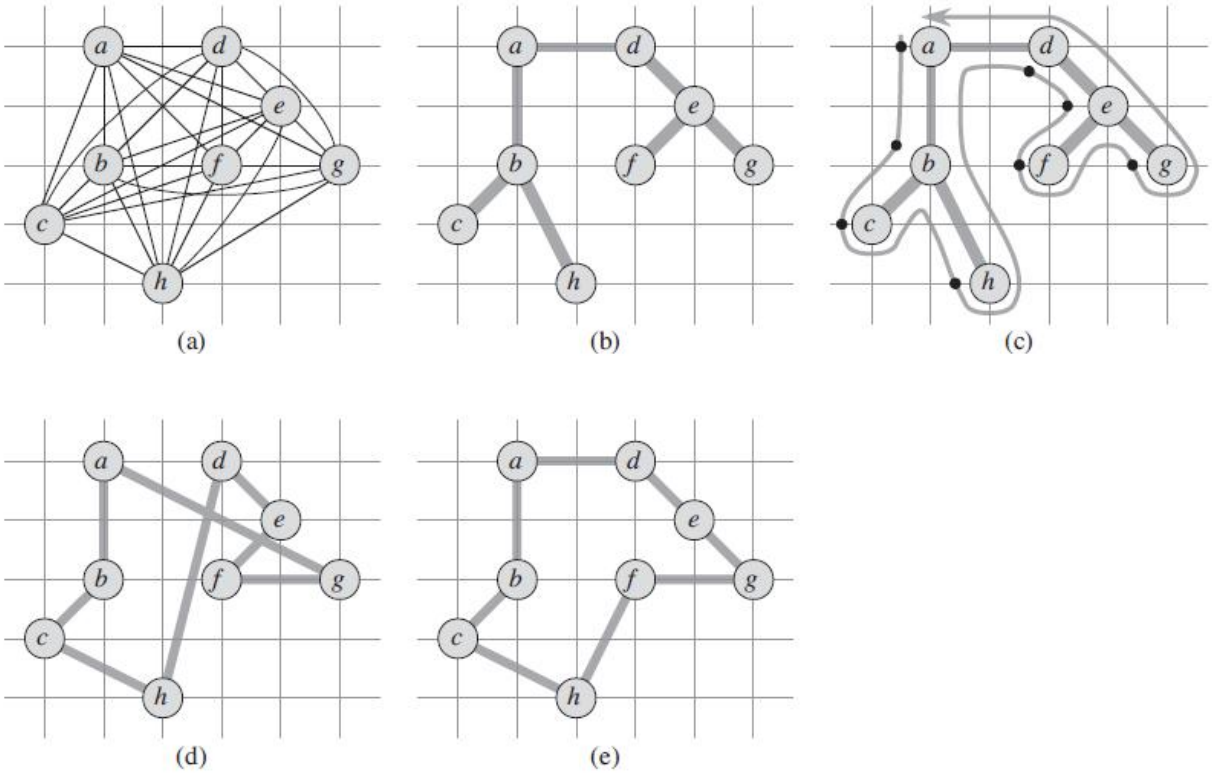
```
Our Tour ≤ 2 × Optimal
```

অর্থাৎ আমাদের tour optimal-এর সর্বোচ্চ দ্বিগুণ।

উদাহরণ: ৭০ কেজিকে ১০ কেজিতে round করা হলে আসল ওজন $65 \le$ ওজন < 75।

TSP-তে এর ব্যবহার: MST-র ওজন optimal tour-এর **lower bound**। আমরা optimal জানি না, কিন্তু এটা জানি যে optimal অন্তত MST-র সমান। এই বাউন্ড দিয়েই ratio প্রমাণ হবে।

Slide 51–53: ছবিসহ উদাহরণ



৮টা বিন্দু (a থেকে h) গ্রিডের উপর আছে।

- **(a)** পুরো complete গ্রাফ
- **(b)** এর MST, root = a
- **(c)** MST-র চারপাশে ঘুরে full walk: a, b, c, b, h, b, a, d, e, f, e, g, e, d, a
- **Pre-order walk** (প্রথমবার দেখামাত্র লেখা): **a, b, c, h, d, e, f, g**
- **(d)** এই ক্রমে ঘুরলে আমাদের tour H পাওয়া যায়, cost ≈ **19.074**
- **(e)** আসল optimal tour, cost ≈ **14.715**

Ratio: 19.074 / 14.715 ≈ **1.30**

মানে এই উদাহরণে আমাদের উত্তর optimal-এর চেয়ে মাত্র ৩০% বেশি, যা 2-এর সীমার অনেক নিচে।

Slide 54–56: প্রমাণ (পরীক্ষায় গুরুত্বপূর্ণ)

Theorem: Triangle inequality থাকলে APPROX-TSP-TOUR একটা **Polynomial time 2-approximation algorithm।**

এটাই পরীক্ষার সবচেয়ে গুরুত্বপূর্ণ অংশ।

আমাদের প্রমাণ করতে হবে:

$$c(H) \leq 2c(H^*)$$

যেখানে:

- H = আমাদের বানানো tour
- H^* = optimal tour

Step 1: Algorithm polynomial

আমরা:

1. MST বের করি
2. Pre-order walk করি
3. Tour বানাই

MST Prim দিয়ে polynomial time-এ পাওয়া যায়।

তাই algorithm:

Polynomial Time

Step 2: MST ≤ Optimal Tour

Optimal tour:

H^*

থেকে একটা edge বাদ দিলে একটা spanning tree পাওয়া যায়।

MST যেহেতু সবচেয়ে কম cost-এর spanning tree:

```
c(T) ≤ c(H*)
```

এটাই প্রথম গুরুত্বপূর্ণ equation।

সহজ ভাষায়:

MST কখনো optimal TSP tour-এর চেয়ে বেশি খরচের হবে না।

Step 3: MST-এর প্রতিটি edge দুইবার হাঁটি

MST-তে যদি আমরা পুরো DFS walk করি, তাহলে:

- যাওয়ার সময় edge ব্যবহার করি
- আবার ফিরে আসার সময় একই edge ব্যবহার করি

তাই প্রতিটি edge **২ বার** ব্যবহৃত হয়।

যদি:

```
MST cost = 10
```

তাহলে full walk-এর cost:

```
2 × 10 = 20
```

অর্থাৎ:

```
c(W) = 2c(T)
```

Step 4: Full Walk-এর cost কত?

আমরা জানি:

```
c(T) ≤ c(H*)
```

তাই দুই পাশে 2:

```
2c(T) ≤ 2c(H*)
```

আর:

```
c(W) = 2c(T)
```

তাই:

```
c(W) ≤ 2c(H*)
```

অর্থাৎ:

Full walk optimal-এর সর্বোচ্চ 2 গুণ।

🔴 কিন্তু Full Walk তো TSP Tour নয়!

এখানে সমস্যা আছে।

Full walk-এ:

```
b
e
d
```

একাধিকবার এসেছে।

কিন্তু TSP-তে:

প্রতিটি vertex ঠিক একবার আসতে হবে।

তাই Full Walk সরাসরি answer না।

🔴 Step 5: Shortcut করি

Repeated vertex বাদ দিই।

যেমন:

```
A → B → C → B → D
```

এখানে দ্বিতীয় B বাদ দিয়ে:

```
A → B → C → D
```

করলাম।

Triangle inequality থাকার কারণে:

```
cost(A→C→D)
```

এর চেয়ে shortcut করে যাওয়ার cost বেশি হবে না।

অর্থাৎ:

```
c(H) ≤ c(W)
```

⭐ এখন তিনটা equation একসাথে

আমরা জানি:

1 MST ≤ Optimal
```
c(T) ≤ c(H*)
```
2 Full Walk = 2 × MST
```
c(W) = 2c(T)
```
3 Shortcut করার পর cost বাড়ে না
```
c(H) ≤ c(W)
```

তাই:

```
c(H)
≤ c(W)
= 2c(T)
≤ 2c(H*)
```

অর্থাৎ:

⭐ **c(H) ≤ 2c(H*)**

এটাই প্রমাণ করে:

APPROX-TSP-TOUR হলো 2-Approximation Algorithm।

আমাদের tour optimal-এর সর্বোচ্চ ২ গুণ। এটাই 2-approximation।

যুক্তিটা এক নজরে

```
MST-র ওজন  ≤  Optimal tour          (lower bound)
Full walk   =  2 × MST-র ওজন         (প্রতি edge দুবার)
Our tour    ≤  Full walk             (shortcut, triangle inequality)
──────────────────────────────────────
Our tour ≤ 2 × Optimal tour
```

দৈনন্দিন উদাহরণ: সব গ্রাম ঘুরতে সবচেয়ে সস্তা রাস্তার নকশা (MST) 10 কিমি। ওই নকশা ধরে সব গ্রামে গিয়ে একই রাস্তায় ফিরে এলে 20 কিমি লাগে। শর্টকাট নিলে সেটা আরও কমে যায় (triangle inequality)। আর সেরা tour কখনো 10 কিমির কম হতে পারে না। তাই আমরা সর্বোচ্চ 2 গুণ খারাপ।

Slide 57–63: Bin Packing (সহজ বাংলায়)

🟤 Bin Packing Problem কী?

ধরো তোমার কাছে কিছু ব্যাগ (Bin) আছে।

প্রতিটি ব্যাগে সর্বোচ্চ:

```
10 কেজি
```

ধারণক্ষমতা আছে।

এখন তোমার কাছে কিছু জিনিস আছে:

```
4, 8, 1, 4, 2, 1
```

প্রশ্ন হলো:

সব জিনিস রাখতে সবচেয়ে কম কয়টা ব্যাগ লাগবে?

এটাই **Bin Packing Problem**।

🎒 সহজ উদাহরণ

ধরি:

```
Capacity = 10
Items = 4, 8, 1, 4, 2, 1
```

আমরা এভাবে রাখতে পারি:

```
Bag 1 = 4 + 4 + 2 = 10
Bag 2 = 8 + 1 + 1 = 10
```

মোট:

```
2 Bag
```

এটাই Optimal Solution।

🟤 কেন কঠিন?

যদি 100টা item থাকে, তাহলে কোন item কোন bag-এ যাবে, এটা বের করার অনেক possibility আছে।

তাই:

Exact answer বের করা কঠিন (NP-Hard)।

সেজন্য Approximation Algorithm ব্যবহার করি।

Slide 59: Lower Bound (কমপক্ষে কয়টা bin লাগবেই)

Lower Bound মানে:

কমপক্ষে কতগুলো Bin লাগবেই।

Formula:

```
Lower Bound
= ceil(Total Item Size / Bin Capacity)
```

Example

Items:

```
4,8,1,4,2,1
```

Total:

```
20
```

Bin capacity:

```
10
```

তাই:

```
20/10 = 2
```

অর্থাৎ:

কমপক্ষে `2 Bin` লাগবেই।

আরেকটা Example

Items:

```
9,8,2,2,5,4
```

Total:

```
30
```

Capacity:

```
10
```

তাই:

```
30/10 = 3
```

Lower Bound:

```
3
```

কিন্তু actual optimal:

```
4
```

কেন?

কারণ item কীভাবে একসাথে বসবে সেটাও গুরুত্বপূর্ণ।

যেমন:

```
9 + 2 = 11 ✗
```

তাই 9-এর সাথে 2 রাখা যাচ্ছে না।

☆ মনে রাখবে

Lower Bound exact answer না-ও হতে পারে।

এটা শুধু বলে:

"এর চেয়ে কম Bin-এ কোনোভাবেই হবে না।"

Slide 60: Applications ও Online Algorithm

ব্যবহার: ট্রাকে মাল তোলা, একাধিক ডিস্কে ডেটা রাখা, job scheduling, রেডিও/টিভির নির্দিষ্ট বিজ্ঞাপন-বিরতিতে বিজ্ঞাপন সাজানো, টেপ/CD-তে গান রাখা।

Online Algorithm:

এখানে item একসাথে সব আসে না।

একটা একটা করে আসে।

যেমন:

প্রথমে → `4`
তারপর → `8`
তারপর → `1`
তারপর → `4`
`...`

তুমি আগে থেকে জানো না পরের item কী হবে।

তাই item আসার সাথে সাথে সিদ্ধান্ত নিতে হবে:

এই item কোন Bin-এ রাখব?

এটাই **Online Algorithm**।

Next Fit (স্লাইডের প্রথম online algorithm):

Next Fit-এর rule খুব সহজ:

শুধু সর্বশেষ Bin দেখবে।

যদি সেখানে item ঢোকে: সেখানেই রাখবে।

যদি না ঢোকে: নতুন `Bin` খুলবে।

আগের Bin-এ আর ফিরে যাবে না।

Example

Capacity:

```
10
```

Items:

```
4,8,1,4,2,1
```

Step 1

4:

```
Bin 1 = [4]
```

Step 2

8:

`4+8 = 12` ✗

তাই নতুন Bin:

```
Bin 1 = [4]
Bin 2 = [8]
```

Step 3

1:

`8+1 = 9` ☑

তাই:

`Bin 2 = [8,1]`

Step 4

4:

`9+4 = 13` ✗

নতুন Bin:

`Bin 3 = [4]`

Step 5

2:

`4+2 = 6` ☑

তাই:

`Bin 3 = [4,2]`

Step 6

1:

`6+1 = 7` ☑

Final:

```
Bin 1 = [4]
Bin 2 = [8,1]
Bin 3 = [4,2,1]
```

তাই:

```
Next Fit = 3 Bin
```

কিন্তু optimal ছিল:

```
2 Bin
```

সমস্যা কোথায়?

Bin 1-এ 6 capacity খালি ছিল।

কিন্তু Next Fit বলছে:

"আমি শুধু শেষ Bin দেখব।"

তাই Bin 1-এ আর ফিরে যায়নি।

Slide 61: সংজ্ঞা (সাইজ 1 ধরে নিয়ে)

এই slide-এ Bin-এর capacity সহজ করার জন্য:

```
Capacity = 1
```

ধরা হয়েছে।

Item-এর size হবে:

```
0 < size ≤ 1
```

এখানে:

OPT(I)

মানে:

Optimal solution-এ যতগুলো Bin লাগে।

ধরি:

```
OPT(I) = 3
```

মানে minimum 3 Bin দরকার।

গুরুত্বপূর্ণ Lower Bound

সব item-এর size যোগ করলে:

```
Total Size
```

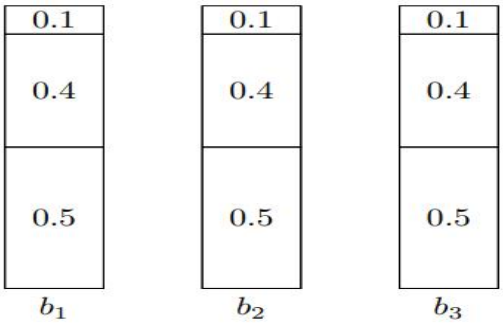
পাব।

যেহেতু প্রতিটি Bin-এর capacity 1:

```
Total Size ≤ OPT(I)
```

অর্থাৎ:

মোট item size যত, তার চেয়ে কম Bin কখনো লাগতে পারে না।



Example

```
S = {0.5,0.1,0.1,0.1,0.5,0.4,0.5,0.4,0.4}
```

Total:

```
0.5+0.1+0.1+0.1
+0.5+0.4
+0.5+0.4+0.4

= 3
```

তাই:

```
OPT ≥ 3
```

আবার 3 Bin-এ রাখা যায়:

```
Bin 1 = 0.5+0.4+0.1 = 1
Bin 2 = 0.5+0.4+0.1 = 1
Bin 3 = 0.5+0.4+0.1 = 1
```

তাই:

```
OPT = 3
```

Slide 62: First Fit Algorithm

এখন First Fit।

এটা Next Fit থেকে একটু আলাদা।

Next Fit:

শুধু শেষ Bin দেখে।

First Fit:

প্রথম Bin থেকে শুরু করে একটার পর একটা দেখে।

যে Bin-এ item প্রথমবার fit করবে, সেখানে রাখবে।

Example

Capacity:

```
10
```

Items:

```
4,8,1,4,2,1
```

Step 1

4:

```
Bin 1 = [4]
```

Step 2

8:

Bin 1-এ:

```
4+8 = 12
```

❌

তাই:

```
Bin 2 = [8]
```

Step 3

1:

Bin 1 check:

```
4+1 = 5
```

☑

তাই:

```
Bin 1 = [4,1]
```

Step 4

4:

Bin 1:

```
4+1+4 = 9
```

☑

তাই:

```
Bin 1 = [4,1,4]
```

Step 5

2:

Bin 1:

```
9+2 = 11
```

❌

তাই Bin 2 দেখি:

```
8+2 = 10
```

☑

তাই:

```
Bin 2 = [8,2]
```

Step 6

1:

আবার Bin 1 দেখি:

```
9+1 = 10 ☑
```

তাই:

```
Bin 1 = [4,1,4,1]
```

Final:

```
Bin 1 = [4,1,4,1]
Bin 2 = [8,2]
```

তাই:

```
First Fit = 2 Bin
```

এবার optimal-ও 2।

⭐ Next Fit বনাম First Fit

এটা পরীক্ষায় খুব সহজে আসতে পারে।

Next Fit	First Fit
শুধু শেষ Bin দেখে	প্রথম থেকে সব Bin দেখে
আগের Bin-এ ফিরে যায় না	আগের Bin-এ ফিরে যায়
নতুন Bin তুলনামূলক বেশি হতে পারে	সাধারণত বেশি জায়গা ব্যবহার করতে পারে
সহজ	একটু বেশি checking লাগে

মনে রাখার Trick:

Next = শুধু Next/শেষ Bin

First = First suitable Bin

Slide 63: First Fit একটা 2-Approximation (প্রমাণ)

এটাই এই অংশের সবচেয়ে গুরুত্বপূর্ণ slide।

আমাদের প্রমাণ করতে হবে:

$$\text{First Fit} \leq 2 \times \text{OPT}$$

অর্থাৎ First Fit কখনো optimal-এর দ্বিগুণের বেশি Bin ব্যবহার করবে না।

⋆ Lemma: সর্বোচ্চ একটা Bin অর্ধেকের কম ভরা থাকতে পারে

ধরো capacity:

```
1
```

এখন যদি দুইটা Bin থাকে:

```
Bin 1 = 0.4
Bin 2 = 0.3
```

দুটোই অর্ধেকের কম।

তাহলে Bin 2-এর itemগুলো Bin 1-এ ঢোকার সুযোগ থাকতে পারে।

First Fit তো আগে Bin 1 check করে।

তাহলে Bin 2 কেন খুলবে?

এটাই contradiction।

তাই:

First Fit-এ দুইটা Bin একসাথে half-empty থাকতে পারে না।

অর্থাৎ:

সর্বোচ্চ **একটা Bin** অর্ধেক বা তার কম ভরা থাকতে পারে।

⭐ Theorem: First Fit হলো 2Approximation

ধরি First Fit:

```
m Bin
```

ব্যবহার করেছে।

যেহেতু সর্বোচ্চ একটা Bin অর্ধেকের কম হতে পারে,

তাই বাকি:

```
m - 1
```

টা Bin **অর্ধেকের বেশি ভরা।**

তাই মোট item size:

```
Total Size > (m-1)/2
```

আবার Lower Bound থেকে:

```
Total Size ≤ OPT
```

তাই:

```
(m-1)/2 < OPT
```

দুই পাশে 2:

```
m - 1 < 2OPT
```

তাই:

```
m ≤ 2OPT
```

⭐ Final:

```
First Fit ≤ 2 × OPT
```

তাই:

First Fit একটি 2-Approximation Algorithm।

অবশ্যই। এই proof-টা একটা **খুব সহজ বাস্তব example** দিয়ে বুঝি।

ধরি:

প্রতিটি Bin-এর capacity = **10**

First Fit মোট **m = 4টা Bin** ব্যবহার করেছে।

ধরি packing হলো:

```
B1 = 8
B2 = 7
B3 = 6
B4 = 4
```

এখানে প্রথম 3টা Bin অর্ধেকের বেশি ভরা:

```
B1 = 8  > 5
B2 = 7  > 5
B3 = 6  > 5
B4 = 4  < 5
```

অর্থাৎ **সর্বোচ্চ ১টা Bin** অর্ধেকের কম হতে পারে।

Step 1: মোট কতটা item আছে?

```
Total Size = 8 + 7 + 6 + 4
           = 25
```

আর আমাদের formula ছিল:

```
Total Size > (m-1)/2 × Capacity
```

এখানে:

```
m = 4
Capacity = 10
```

তাই:

```
Total Size > (4-1)/2 × 10
           > 3/2 × 10
           > 15
```

সত্যিই:

```
25 > 15
```

Step 2: OPT কী বলে?

ধরি এই items-গুলো রাখার জন্য **OPT = 3 Bin** যথেষ্ট।

তাহলে total capacity হবে:

```
3 × 10 = 30
```

তাই মোট item size কখনোই 30-এর বেশি হতে পারে না।

অর্থাৎ:

```
Total Size ≤ OPT × Capacity
```

Normalized capacity 1 ধরলে:

```
Total Size ≤ OPT
```

তাই আমাদের proof-এ:

```
Total Size ≤ OPT
```

Step 3: দুইটা inequality একসাথে

আমরা পেলাম:

```
(m-1)/2 < OPT
```

এখন 2 দিয়ে গুণ:

```
m - 1 < 2OPT
```

অর্থাৎ:

$$m < 2OPT + 1$$

যেহেতু m এবং OPT integer:

$$m \leq 2OPT$$

একদম সহজ ভাষায় 🙂

First Fit যদি **4টা Bin** ব্যবহার করে:

$$m = 4$$

তাহলে theorem বলছে:

$$4 \leq 2 \times OPT$$

অর্থাৎ:

$$OPT \geq 2$$

মানে আসল optimal solution যদি 2টা Bin হয়, First Fit সর্বোচ্চ:

$$2 \times 2 = 4 \text{ Bin}$$

ব্যবহার করতে পারে।

তাই মূল কথাটা:

First Fit কখনোই OPT-এর 2 গুণের বেশি Bin ব্যবহার করবে না।

যেমন:

$$OPT = 2$$
$$\downarrow$$
$$\text{First Fit} \leq 4$$

$$OPT = 5$$
$$\downarrow$$
$$\text{First Fit} \leq 10$$

$$OPT = 10$$
$$\downarrow$$

$\text{First Fit} \leq 20$

এই কারণেই:

$\star\ \text{First Fit} \leq 2 \times \text{OPT}$

তাই First Fit হলো 2-Approximation Algorithm।